

CS 201 - Data Structures II

Unit 1 – Abstract Data Types (ADTs)

Spring 2024

Habib University

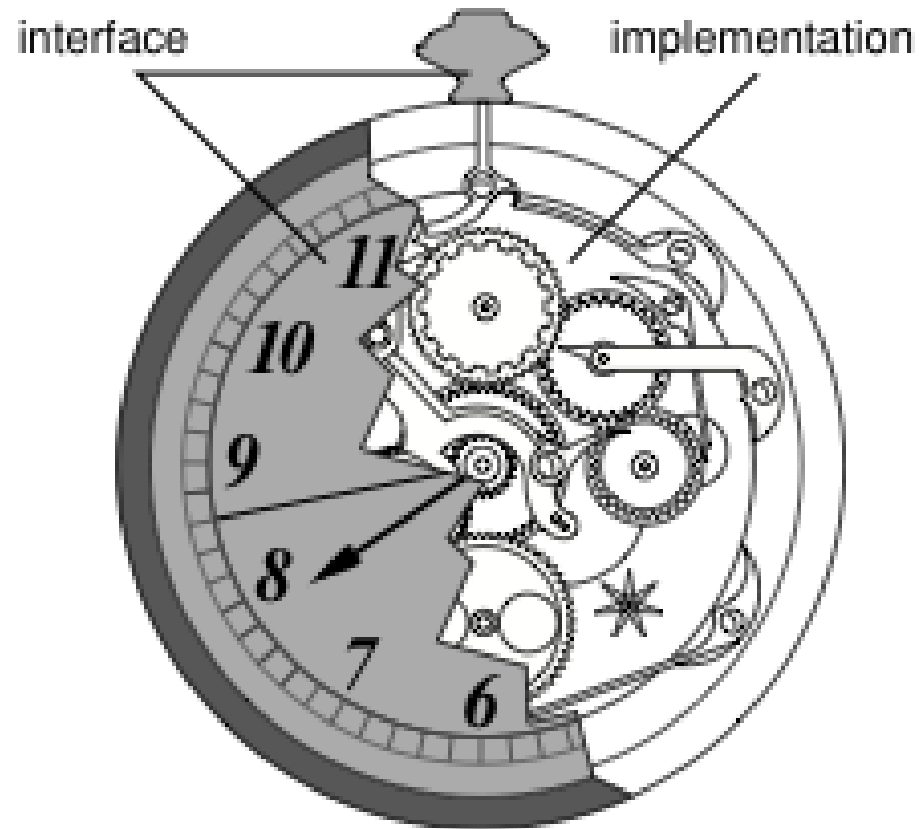
Syeda Saleha Raza

Outline

- Some abstract data types
 - Stack
 - Queue
 - DEQUE
 - List
 - UnorderedSet
 - OrderedSet
- Recap
 - about Arrays
 - Asymptotic Analysis

Abstraction

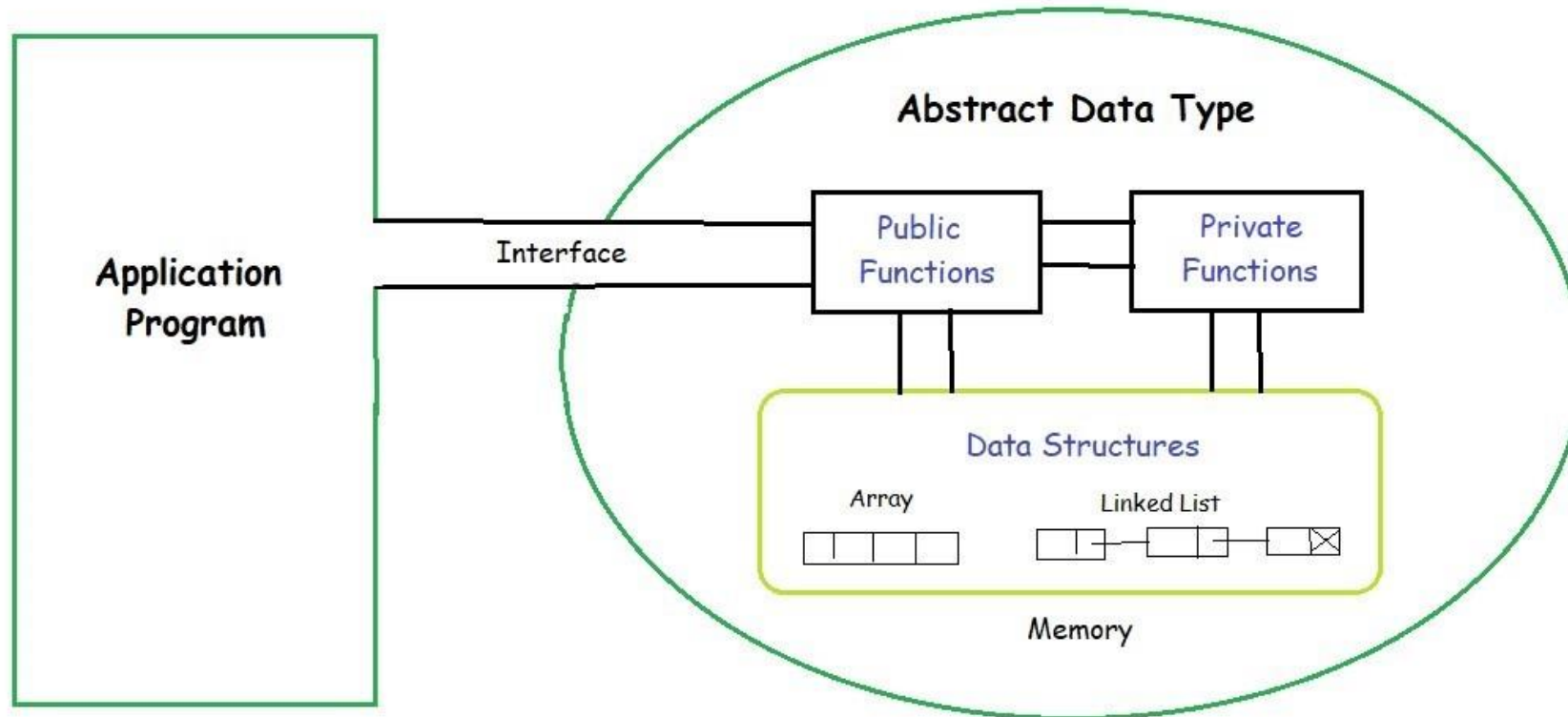
Abstraction



Abstraction

- **Abstraction** is the concept of exposing only the required essential characteristics and behavior with respect to a context.
- Hiding of data is known as **data abstraction**.

Abstract Data Types



<https://www.geeksforgeeks.org/abstract-data-types>

Some Abstract Data Types

- Stack
- Queue
- DEQUE
- List
- UnorderedSet (USet)
- Sorted Set (SSet)

Exercise:

How to implement Stack and Queue using List interface?

Exercise:

How to create a DEQUEUE using two stacks?

Important Considerations

- Linear vs Non-linear
- Sequential vs random access
- Static vs Dynamic
- Homogenous vs heterogeneous
- Time and Space Complexity

Recap

- Arrays
- Big O Notation

Arrays

Arrays

- Static/Fixed size
- Contiguous memory allocation
 - Pros and cons of contiguous memory allocation?
 - large chunks of them are loaded into the cache upon first access. This makes it comparatively quick to access future elements of the array.

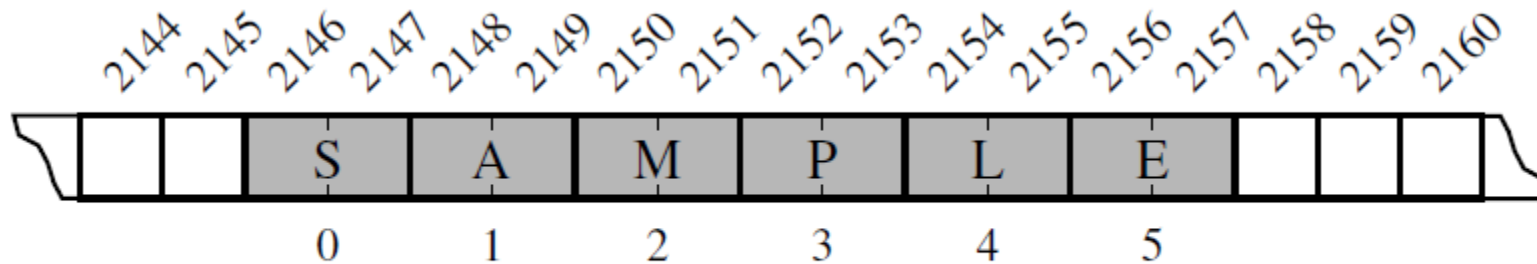


Figure 5.2: A Python string embedded as an array of characters in the computer's

Arrays

- Direct indexing
 - Why?
- Homogenous
 - Why?
- Time complexity: get, search, add, delete?

Model of Computation

- We use the w -bit word-RAM model as our model of computation.
- In the word-RAM model, basic operations on words take constant time.
- This includes arithmetic operations ($+$, $-$, $*$, $/$, $\%$), comparisons ($=$, $\leq$, $\geq$), and bitwise boolean operations (bitwise-AND, OR, and exclusive-OR).
- Any cell can be read or written in constant time.
- Allocating a block of memory of size k takes $O(k)$ time and returns a reference (a pointer) to the newly-allocated memory block.

Asymptotic Analysis - Recap

Complexity Analysis

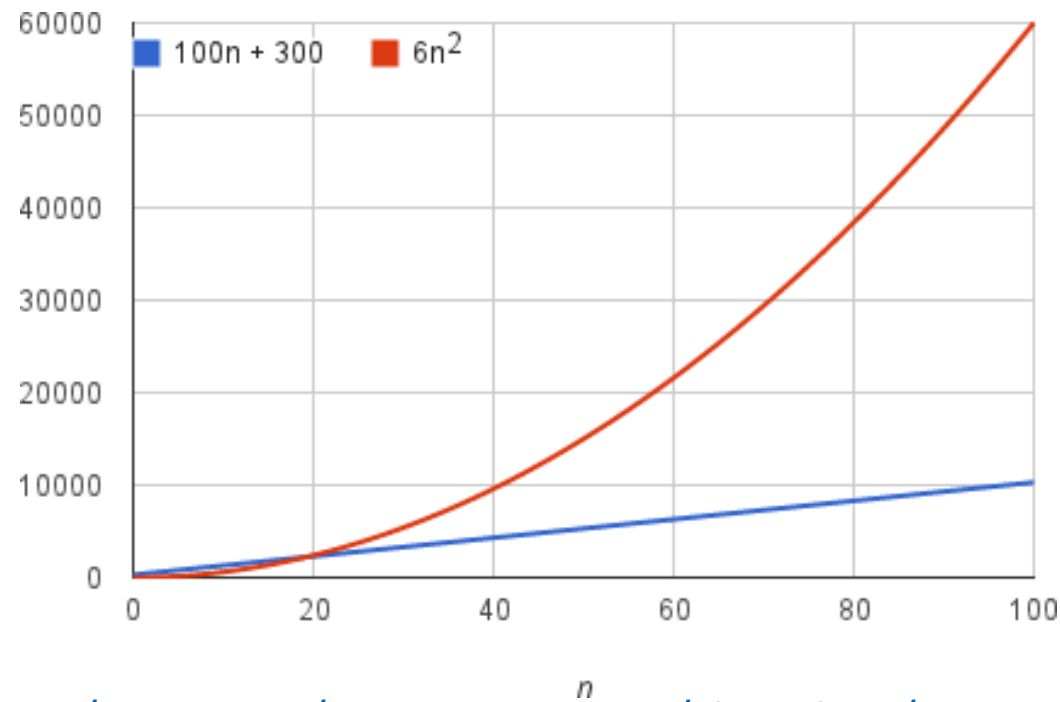
- When we are trying to find the complexity of the function/ procedure/ algorithm/ program, we are not interested in the exact number of operations that are being performed. Instead, we are interested in the relation of the number of operations to the problem size.

Example

```
a=5
b=6
c=10
for i in range(n):
    for j in range(n):
        x = i * i
        y = j * j
        z = i * j
    for k in range(n):
        w = a*k + 45
        v = b*b
d = 33
```

Asymptotic Analysis

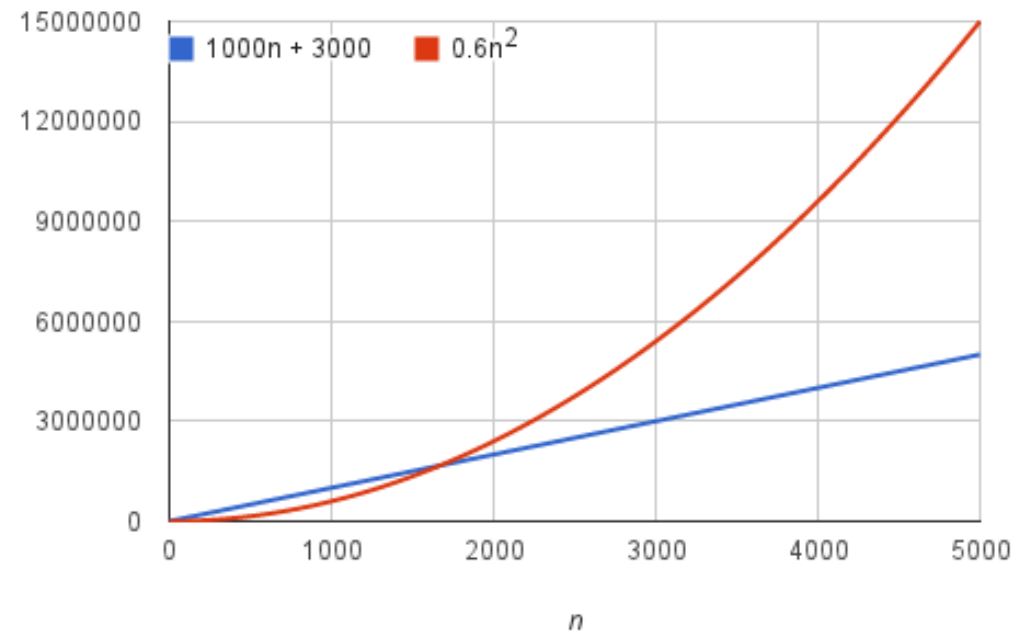
For example, suppose that an algorithm, running on an input of size n takes $6n^2 + 100n + 300$. $6n^2 + 100n + 300$
The $6n^2$ term becomes larger than the remaining terms, $100n + 300$ once n becomes large enough, 20 in this case.



- <https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Asymptotic Analysis

- It doesn't really matter what coefficients we use; as long as the running time is $an^2 + bn + c$ for some numbers a , b , and c , there will always be a value of n for which an^2 is greater than $bn + c$ and this difference increases as n increases.



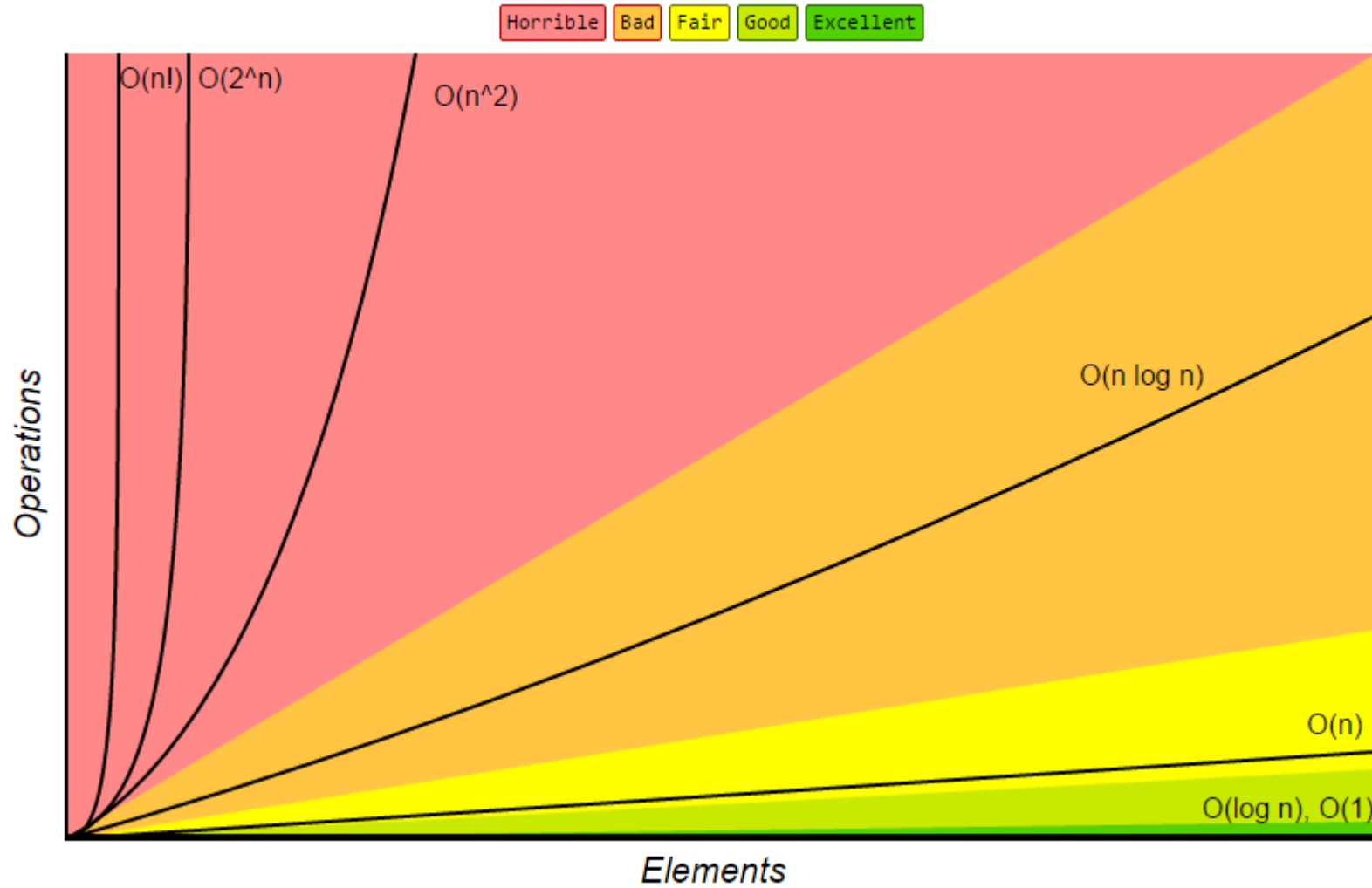
<https://www.khanacademy.org/computing/computer-science/algorithms/asymptotic-notation/a/asymptotic-notation>

Asymptotic Analysis

- By dropping the less significant terms and the constant coefficients, we can focus on the important part of an algorithm's running time, — its rate of growth. When we drop the constant coefficients and the less significant terms, we use **asymptotic notation**.

1. $\Theta(1)$
2. $\Theta(\lg n)$
3. $\Theta(n)$
4. $\Theta(n \lg n)$
5. $\Theta(n^2)$
6. $\Theta(n^2 \lg n)$
7. $\Theta(n^3)$
8. $\Theta(2^n)$

Big-O Complexity Chart



<http://bigocheatsheet.com/>

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at <http://opendatastructures.org>.
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing.

Thanks